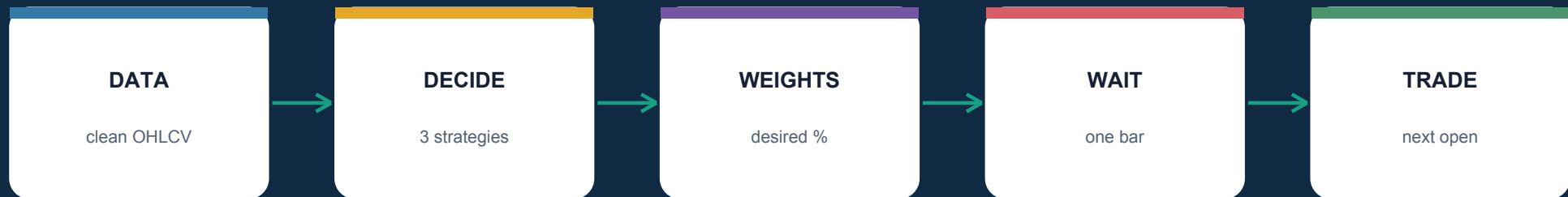


Understand the Strategy Brain

Every file. Every function. Real code. Tiny examples. Interview reasoning.

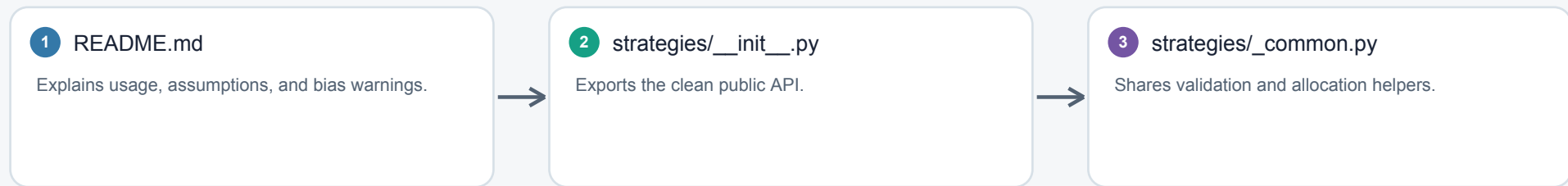


THE CORE INTERVIEW IDEA

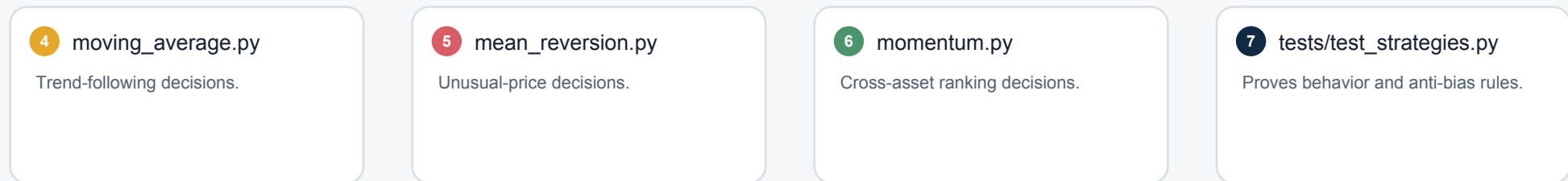
Strategies decide what to own. The engine decides how and when to trade.

Seven Phase 4 files and their jobs

Follow the arrows: documentation -> public API -> strategies -> engine -> tests.



THREE IMPLEMENTATIONS



Interaction: all three strategies call `_common.py`, return the same target-weight shape, and plug into the existing Phase 3 Backtester without knowing anything about cash or orders.

validated_close_prices(): build one trusted input

Input: symbol -> OHLCV DataFrame. Output: Date x Symbol closing-price DataFrame.

ACTUAL LOGIC, SHORTENED ONLY FOR ERROR TEXT

```
1 def validated_close_prices(market_data):
2     if not market_data:
3         raise StrategyError(...)
4     normalized_data = {}
5     reference_index = None
6     for raw_symbol, data in market_data.items():
7         symbol = normalize_symbol(raw_symbol)
8         if symbol in normalized_data:
9             raise StrategyError(...)
10        validate_ohlc(data)
11        if reference_index is None:
12            reference_index = data.index
13        elif not data.index.equals(reference_index):
14            raise StrategyError(...)
15        normalized_data[symbol] = data
16    symbols = sorted(normalized_data)
17    return pd.DataFrame({
18        s: normalized_data[s]["Close"].astype(float)
19        for s in symbols
20    }, index=reference_index)
```

1 1. Normalize

"aapl" becomes "AAPL" so naming is consistent.

2 2. Validate

Reject missing columns, impossible prices, bad dates, and duplicate normalized symbols.

3 3. Align

Every asset must use exactly the same timestamps, or rows would compare different days.

4 StrategyError + result

Bad input raises one specific error type; valid input becomes sorted float closes.

Two helpers remove repeated mistakes

Equal allocation handles active assets; integer validation protects every configuration.

EQUAL WEIGHTS

```
1 def equal_weight_active_positions(active):
2     active_counts = active.sum(axis=1).astype(float)
3     divisors = active_counts.where(
4         active_counts > 0, float("nan")
5     )
6     weights = active.astype(float).div(
7         divisors, axis=0
8     ).fillna(0.0)
9     return weights.astype(float)
```

SAFE SETTINGS

```
1 def validate_positive_integer(value, label):
2     if (
3         isinstance(value, bool)
4         or not isinstance(value, Integral)
5         or value <= 0
6     ):
7         raise StrategyError(
8             f"{label} must be a positive integer."
9         )
10    return int(value)
```

Active?	AAPL	MSFT	Count	Weights
Row 1	True	True	2	0.5 / 0.5
Row 2	False	False	0	0.0 / 0.0

Why NaN?

When count is zero, divide-by-zero becomes NaN, then `fillna(0.0)` safely means hold cash.

`__init__.py` and `README.md`: the front door

These files make the strategy package easy to discover, import, and defend.

WHAT A USER IMPORTS

```
1 from samquant.strategies import (  
2     MeanReversionStrategy,  
3     MomentumStrategy,  
4     MovingAverageCrossoverStrategy,  
5     StrategyError,  
6 )
```

WHAT THE PACKAGE PROMISES

```
1 __all__ = [  
2     "MeanReversionStrategy",  
3     "MomentumStrategy",  
4     "MovingAverageCrossoverStrategy",  
5     "StrategyError",  
6 ]
```

1 Clean import

Users do not need to know each strategy's internal file path.

2 Encapsulation

`_common.py` begins with an underscore: it is an internal implementation detail, not the public API.

3 README contract

Shows data -> strategy -> weights -> backtester and states the next-open timing rule.

4 Honesty

No profitability promise; mean reversion and survivorship assumptions are named.

Configuration: an immutable strategy object

Defaults model the classic 50-day versus 200-day trend comparison.

moving_average.py / `__post_init__`

```
1 @dataclass(frozen=True)
2 class MovingAverageCrossoverStrategy:
3     short_window: int = 50
4     long_window: int = 200
5
6     def __post_init__(self):
7         short = validate_positive_integer(
8             self.short_window, "Short window"
9         )
10        long = validate_positive_integer(
11            self.long_window, "Long window"
12        )
13        if short >= long:
14            raise StrategyError(...)
15        object.__setattr__(self, "short_window", short)
16        object.__setattr__(self, "long_window", long)
```

1 @dataclass

Python creates the constructor and readable representation automatically.

2 frozen=True

Parameters cannot accidentally change halfway through a backtest.

3 __post_init__

Runs immediately after construction and rejects short >= long, zero, decimals, and booleans.

MovingAverageCrossoverStrategy(2, 3) -> valid research settings

MOVING AVERAGE

Algorithm: wait, compare, allocate

Input: symbol -> OHLCV. Output: Date x Symbol weights. A short trend above a long trend becomes active.

generate_target_weights()

```
1 close = validated_close_prices(market_data)
2 short_avg = close.rolling(
3     window=self.short_window,
4     min_periods=self.short_window,
5 ).mean()
6 long_avg = close.rolling(
7     window=self.long_window,
8     min_periods=self.long_window,
9 ).mean()
10 active = short_avg.gt(long_avg) & long_avg.notna()
11 return equal_weight_active_positions(active)
```

Day	Close	Short(2)	Long(3)	Active	Weight
D1	10	-	-	False	0%
D2	10	10	-	False	0%
D3	10	10	10	False	0%
D4	12	11	10.67	True	100%
D5	14	13	12	True	100%

+ 1 WAIT

min_periods blocks incomplete averages

+ 2 COMPARE

short > long means uptrend

+ 3 ALLOCATE

one active asset gets 100%

+ 4 EXECUTE

engine trades next open

Configuration + z-score: how unusual is today?

The strategy enters after a deep drop and exits when price recovers toward its mean.

CONFIGURATION + NUMBER CHECK

```
1 @dataclass(frozen=True)
2 class MeanReversionStrategy:
3     lookback_window: int = 20
4     entry_z_score: float = -2.0
5     exit_z_score: float = 0.0
6
7     def __post_init__(self):
8         # lookback >= 2
9         # both z-scores must be finite numbers
10        # entry must be smaller than exit
11
12    @staticmethod
13    def _is_finite_number(value):
14        return (
15            not isinstance(value, bool)
16            and isinstance(value, Real)
17            and isfinite(value)
```

$$z = (\text{price} - \text{rolling mean}) / \text{rolling std}$$

Negative = below normal Positive = above normal

Recent prices	10, 10, 8
Mean	9.33
Std	0.94
Today's z	$(8 - 9.33) / 0.94 = -1.41$

1 Entry

If $z \leq$ entry threshold and we are not already long, switch ON.

2 Hold

Remain invested between entry and exit. This memory makes it stateful.

3 Exit

If $z \geq$ exit threshold while long, switch OFF and return to cash.

MEAN REVERSION

Algorithm: a tiny state machine

Input: symbol -> OHLCV. Output: Date x Symbol weights. is_long remembers whether each symbol is active.

generate_target_weights()

```
1 rolling_mean = close.rolling(window, min_periods=window).mean()
2 rolling_std = close.rolling(window, min_periods=window).std(ddof=0)
3 z = (close - rolling_mean) / rolling_std.replace(0.0, np.nan)
4
5 active = DataFrame(False, index=close.index, columns=close.columns)
6 for symbol in close.columns:
7     is_long = False
8     for timestamp, z_score in z[symbol].items():
9         if pd.notna(z_score):
10             if not is_long and z_score <= entry:
11                 is_long = True
12             elif is_long and z_score >= exit:
13                 is_long = False
14         active.at[timestamp, symbol] = is_long
15 return equal_weight_active_positions(active)
```

CASH z = -0.4



ENTER z = -2.2



HOLD z = -1.0



EXIT z = 0.1

+ Zero volatility

std = 0 becomes NaN; pd.notna fails, so no false signal is created.

Configuration + trailing return

Rank assets by their own lookback returns, then select the strongest.

CONFIGURATION + `__post_init__` + SCORE

```
1 @dataclass(frozen=True)
2 class MomentumStrategy:
3     lookback_window: int = 126
4     top_n: int = 1
5     rebalance_frequency: int = 21
6     require_positive_returns: bool = True
7
8     def __post_init__(self):
9         # validate all three integers > 0
10        # require_positive_returns must be bool
11
12    trailing_returns = (
13        close_prices
14        / close_prices.shift(lookback_window)
15        - 1.0
16    )
```

1 lookback_window

How far back to measure performance. 126 is roughly six trading months.

2 top_n

How many winners to own.

3 rebalance_frequency

How many rows to wait before ranking again. 21 is roughly one trading month.

4 require_positive_returns

If every score is negative, hold cash instead of a losing asset.

Algorithm: rank, choose, freeze until rebalance

Input: symbol -> OHLCV. Output: Date x Symbol weights. The loop keeps weights fixed between rebalances.

generate_target_weights()

```
1 for position in range(len(close_prices.index)):
2     should_rebalance = (
3         position >= lookback
4         and (position - lookback) % frequency == 0
5     )
6     if should_rebalance:
7         scores = trailing_returns.iloc[position].dropna().sort_index()
8         if require_positive_returns:
9             scores = scores[scores > 0]
10        winners = scores.sort_values(
11            ascending=False, kind="mergesort"
12        ).index[:top_n]
13        current_weights = Series(0.0, index=symbols)
14        if len(winners) > 0:
15            current_weights.loc[winners] = 1.0 / len(winners)
16        weights.iloc[position] = current_weights
```

Asset	Return	Rank	Weight
AAPL	+20%	1	50%
MSFT	+10%	2	50%
GOOG	-5%	3	0%

1 Stable and safe ranking

NaNs are dropped; top_n may select fewer valid assets; alphabetical mergesort resolves ties consistently.

2 Hold behavior

On non-rebalance days, the previous current_weights are copied forward unchanged.

From strategy object to a real simulated trade

This is the exact contract between Phase 4 decisions and the Phase 3 engine.

USAGE

```
1 from samquant.engine import Backtester
2 from samquant.strategies import MovingAverageCrossoverStrategy
3
4 market_data = {"AAPL": prices}
5 strategy = MovingAverageCrossoverStrategy(
6     short_window=50,
7     long_window=200,
8 )
9 targets = strategy.generate_target_weights(market_data)
10 result = Backtester().run(market_data, targets)
11
12 print(result.trades)
13 print(result.equity_curve)
```

1 MON 4:00 PM

Close = \$12
Strategy creates 100% target



2 ENGINE SHIFT

target_weights.shift(1)
Decision moves to next row



3 TUE 9:30 AM

Open = \$14
Backtester buys at \$14

Never: use Monday's close signal and pretend the trade also happened at Monday's close.

TESTING

Every test answers one dangerous question

Synthetic data is fast, deterministic, and independent of internet providers.

FAKE BUT VALID OHLCV

```
1 def _ohlcw(closes):
2     dates = pd.date_range("2024-01-02", periods=len(closes))
3     return pd.DataFrame({
4         "Open": closes,
5         "High": [p + 1 for p in closes],
6         "Low": [p - 1 for p in closes],
7         "Close": closes,
8         "Volume": [1_000_000] * len(closes),
9     }, index=dates)
```

1 Why not live data?

Yahoo can change, fail, or return different prices. Unit tests must give the same answer every run.

Current result: 39 project tests pass

Test group	What it proves
MA warm-up	No signal before enough history
MA equal weights	Exposure splits and total <= 100%
Future-change test	Earlier signals ignore future prices
MR entry / exit	State turns on and off correctly
MR zero std	Flat prices create no false trade
Momentum ranking	Winner is held until rebalance
Negative momentum	All-negative universe holds cash
Integration	Signal trades exactly one bar later
Invalid config	Bad settings fail immediately
Misaligned dates	Different timelines are rejected

Design choices, finance ideas, and beginner traps

These are the answers interviewers care about after the code works.

DESIGN CHOICES

- 1 Target weights, not orders
- 2 Frozen strategy settings
- 3 Shared validation
- 4 Long-only first
- 5 Deterministic tie-breaking

FINANCE IDEAS

- 1 Trend following
- 2 Mean reversion assumption
- 3 Cross-sectional momentum
- 4 Rebalancing
- 5 Transaction timing

BEGINNER TRAPS

- 1 Same-close execution
- 2 No indicator warm-up
- 3 Divide by zero std
- 4 Re-rank every day
- 5 Weights above 100%

Bias still to solve: momentum research needs a point-in-time asset universe. Testing today's surviving companies in old years creates survivorship bias.

Explain Phase 4 without AI

Use the pitch, then answer the questions aloud without looking at the code.

15-SECOND PITCH

Phase 4 adds three configurable strategies that transform validated historical closing prices into target portfolio weights. The Phase 3 engine shifts each target by one bar and executes at the next open, separating research logic from realistic execution.

1. Why return weights instead of orders?

2. How is same-bar look-ahead prevented?

3. Why does moving average need min_periods?

4. Why is mean reversion stateful?

5. What happens when standard deviation is zero?

6. Why may momentum hold cash?

7. How are two active assets weighted?

8. Why must all timestamps match?

9. Why use mergesort after alphabetical sorting?

10. What survivorship-bias risk remains?

Start with questions 1-3. Explain the reason, then point to the exact code behavior that supports your answer.